



TempEdge-ResGNN for Bitcoin Illicit Transaction Detection Using the Elliptic Transaction Graph

Student: Alireza Yegane**Student ID:** 60957107**Dataset:** Elliptic Bitcoin transaction graph**Framework:** PyTorch Geometric

Short project summary. This project treats Bitcoin illicit transaction detection as a graph node-classification task. The main result is honest: Random Forest is the best overall model on the handcrafted Elliptic features, while TempEdge-ResGNN is the best neural/GNN model in my experiments. The most important analysis point is the strong temporal distribution shift after timestep 43.

Table of contents

- | | |
|--|--------------------------------|
| 1. Motivation and explanation of data/task | 5. Limitations and future work |
| 2. Appropriateness and explanation of model(s) | 6. Conclusion |
| 3. Insights and results | 7. References |
| 4. Comprehensive analysis | |

Abstract / project summary

This project studies Bitcoin illicit transaction detection as a graph machine learning problem using the Elliptic Bitcoin dataset and PyTorch Geometric. Each node represents a Bitcoin transaction, and directed edges represent payment-flow links between transactions. The task is binary node classification on labelled transactions: licit or illicit. Unknown-labelled nodes are kept in the graph as transductive context but excluded from supervised loss and evaluation.

I compare classical feature-based models, a neural feature-only model, standard GNN baselines, and a custom model called TempEdge-ResGNN. The best overall model is Random Forest, with test illicit-F1 of **0.8058**, precision **0.9175**, recall **0.7184**, ROC-AUC **0.9115**, and PR-AUC **0.7820**. Among neural and graph models, TempEdge-ResGNN performs best, with test illicit-F1 **0.5760 ± 0.0672** and PR-AUC **0.6233 ± 0.0516** across five seeds. The main finding is that GNNs improve over plain GCN on this task, but they do not beat a strong Random Forest baseline on the handcrafted Elliptic features. The per-timestep analysis shows that the largest difficulty is temporal distribution shift after timestep 43.

BEST OVERALL

Random Forest

Illicit-F1 0.8058

BEST NEURAL/GNN

TempEdge-ResGNNIllicit-F1 0.5760 ± 0.0672

MAIN CHALLENGE

Temporal shift

Large drop after timestep 43

1. Motivation and explanation of data/task

Bitcoin illicit transaction detection is a good real-world use case for graph machine learning because suspicious behaviour is not only contained in isolated transactions. The surrounding transaction flow matters: where money comes from, where it

moves next, and whether a transaction is connected to other suspicious activity. For this reason, a graph representation is more suitable than treating every transaction as an independent tabular row.

I use the Elliptic Bitcoin dataset. In the PyTorch Geometric version used in this notebook, the graph contains **203,769 transaction nodes**, **234,355 directed edges**, and **165 numerical node features**. The labels are licit, illicit, and unknown. Only **46,564** nodes are labelled: **42,019 licit** and **4,545 illicit**. This means the labelled illicit rate is about **9.76%**, so the task is highly imbalanced. Because of that imbalance, I do not use accuracy as the main metric. The main metrics are illicit precision, illicit recall, illicit-F1, macro-F1, micro-F1, ROC-AUC, and PR-AUC.

The graph ML task is semi-supervised transductive node classification. The full graph is available for message passing, but the loss is only computed on labelled training nodes. Unknown-labelled nodes are kept in the graph because they may still provide useful structural context, but they are excluded from the supervised loss and from evaluation. Treating unknown nodes as licit would create incorrect labels.

The split is temporal rather than random. The model is trained on earlier timesteps and tested on later timesteps. I also carve validation nodes from timesteps **30-34**, so threshold tuning uses a late training-period validation set without touching the test labels. This is more realistic for AML than random splitting, because a real monitoring system has to use past transactions to flag future ones.

For preprocessing, I recover node timesteps from the raw Elliptic feature CSV because the PyG object does not expose them directly in the executed environment. I then create train, validation, and test masks while excluding unknown labels from supervised training and evaluation. Finally, I standardise the 165 node features using `StandardScaler` fitted only on training nodes. The scaler is not fitted on validation or test nodes, because that would leak future-period statistics into preprocessing.

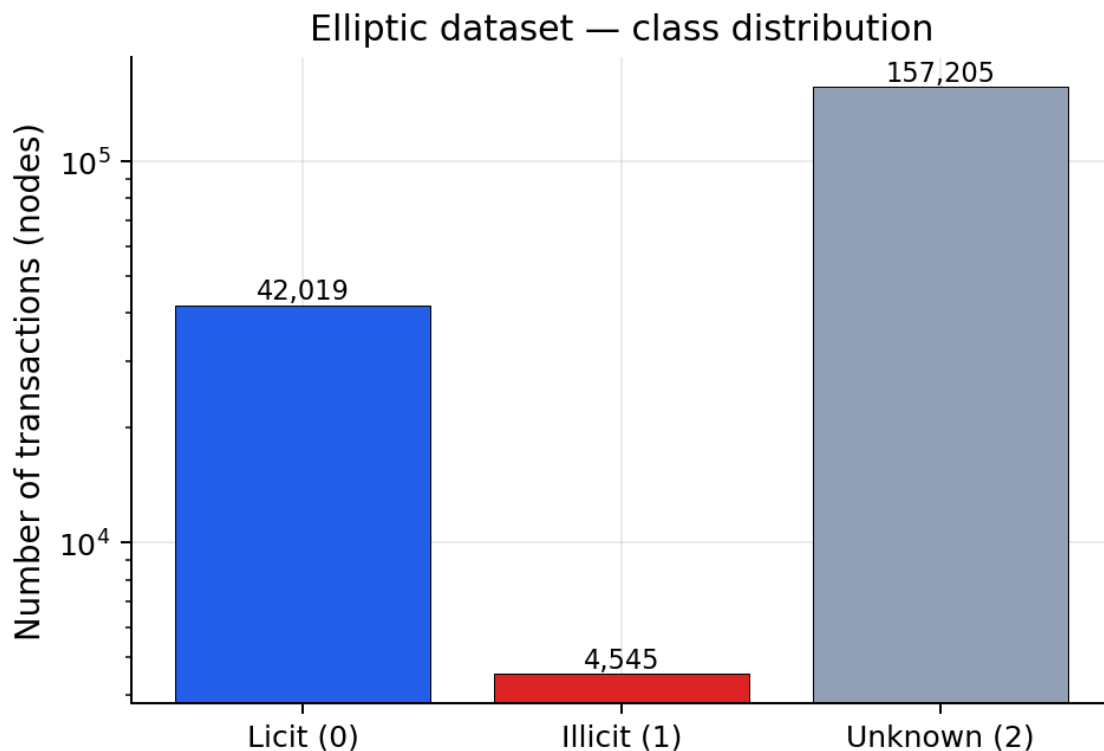


Figure 1. Class distribution in labelled and unknown nodes. The dataset is highly imbalanced, with far fewer illicit than licit labelled transactions.

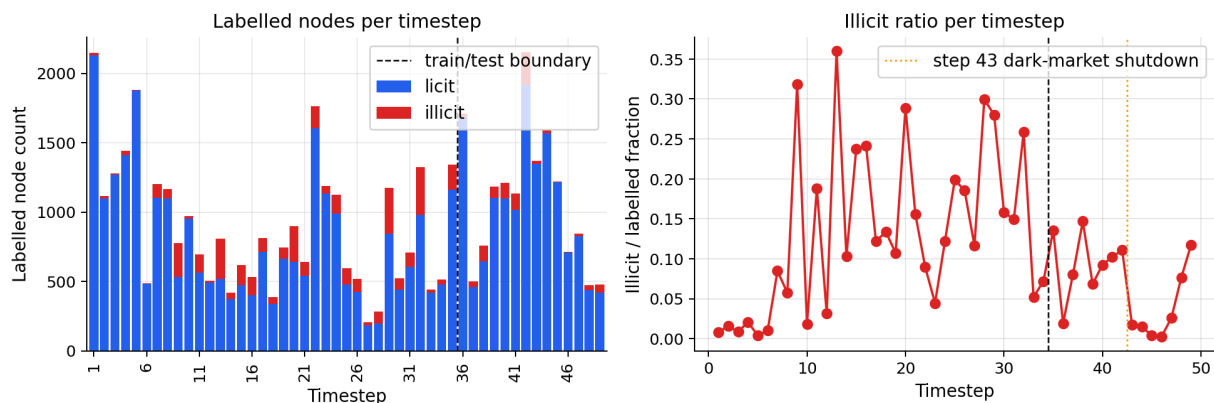


Figure 2. Timestep distribution. The temporal split evaluates whether models trained on earlier transactions generalise to later ones.

Elliptic transaction subgraph used for graph-data illustration
180 nodes, 235 edges; illicit=1, licit=15, unknown=164

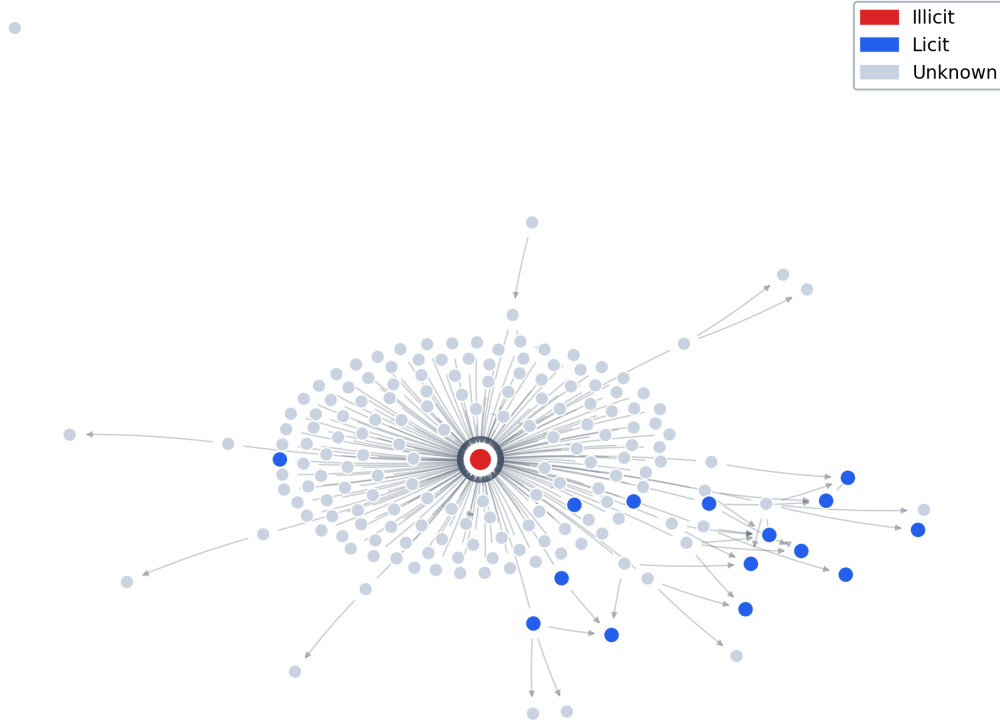


Figure 3. Selected Elliptic transaction subgraph. The figure shows licit, illicit and unknown nodes in the payment-flow graph.

2. Appropriateness and explanation of model(s)

I compare both feature-only and graph-based models because the Elliptic node features are already strong and include local and aggregated transaction statistics. A GNN should therefore be judged against strong non-GNN baselines, not only against weak neural baselines.

The classical baselines are Logistic Regression and Random Forest. Logistic Regression gives a simple linear reference. Random Forest is included because it is a strong model for tabular and engineered features, and previous work on Elliptic also showed that handcrafted features can be very predictive. I also include an MLP because it tests whether a deep model using only node features can compete with graph message passing.

The standard GNN baselines are GCN, GraphSAGE, GAT, and GATv2. GCN is the simplest graph convolution baseline and shows how much is gained from basic neighbourhood smoothing. GraphSAGE tests mean neighbourhood aggregation. GAT and GATv2 test whether attention over neighbours improves over fixed aggregation. These models give a fair comparison because the proposed model is compared against several standard graph architectures, not just one.

My custom model is TempEdge-ResGNN. It uses a custom edge-aware message-passing layer, residual gates, non-parametric timestep features, and optional Jumping Knowledge aggregation. The motivation is practical. Edge-aware message passing allows the model to use engineered structural edge attributes. Residual gates help prevent useful node features from being overwritten by message passing. Non-parametric timestep features, such as t/T , $\sin(2\pi t/T)$, and $\cos(2\pi t/T)$, are used instead of a learned timestep embedding because the test period contains later timesteps. A learned embedding for future timesteps can remain poorly trained, while these fixed time features are defined for every timestep.

One important caveat is that the engineered edge-level temporal feature is limited in this processed PyG graph. The diagnostic output shows `same_step = 100.0%`, meaning all edges connect transactions within the same timestep. Therefore, the edge temporal difference should not be interpreted as a full dynamic transaction-flow signal. The stronger temporal signal in this project comes from node-level timestep features and from the temporal train/validation/test split.

I use weighted BCE as the main neural loss because the labelled data is imbalanced. I also test class-balanced focal loss in the ablation study, but it does not improve the final result, so it is not used as the headline setting.

Method choices in plain language

Why Elliptic? I chose Elliptic because the graph is real, not artificially constructed. Transactions are naturally connected by payment-flow edges, and the task has a clear AML motivation.

Why not a random split? A random split would probably make the numbers easier, but it would not represent deployment. In practice, a model trained on past transactions has to work on future ones.

Why keep unknown nodes? Unknown nodes may still help message passing, but they should not be used as labelled examples. I keep them in the graph but exclude them from the loss and metrics.

Why these baselines? Logistic Regression and Random Forest test the strength of the handcrafted features. MLP tests feature-only deep learning. GCN, GraphSAGE, GAT, and GATv2 test standard GNN message passing.

Why not EvolveGCN, TGN, GraphGPS, or heterogeneous GNNs? Those methods are interesting, but they need a more complex dynamic or heterogeneous pipeline. For this assignment, I wanted a stable PyG pipeline with clear baselines, ablation, and honest analysis.

Why is Random Forest winning still okay? It shows that the Elliptic handcrafted features are very strong. The GNN work is still useful because TempEdge-ResGNN improves over plain GCN and gives a structured analysis of where graph models struggle.

TempEdge-ResGNN — edge-aware messages, residual gates, non-parametric temporal features, JKNet

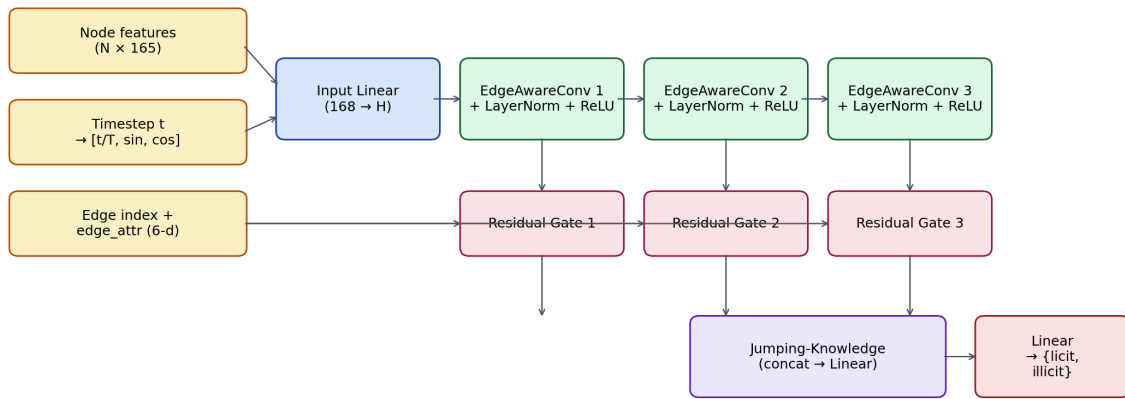


Figure 4. TempEdge-ResGNN architecture. The proposed model combines edge-aware message passing, residual gating, timestep features and JK-style aggregation.

3. Insights and results

The main model comparison is shown in Table 1. Random Forest is the best overall model, reaching test illicit-F1 **0.8058**, precision **0.9175**, recall **0.7184**, ROC-AUC **0.9115**, and PR-AUC **0.7820**. This is an important result because it shows that the handcrafted Elliptic features contain very strong signal.

Among neural and graph models, TempEdge-ResGNN is the best. It reaches test illicit-F1 **0.5760 ± 0.0672** and PR-AUC **0.6233 ± 0.0516** over five seeds. This is better than MLP (**0.4839 ± 0.0138** F1), GCN (**0.4182 ± 0.0208**), GraphSAGE (**0.5033 ± 0.0076**), GAT (**0.5002 ± 0.0210**), and GATv2 (**0.5173 ± 0.0345**). The model therefore improves clearly over plain GCN and the other neural baselines, but it still does not beat Random Forest.

I interpret this result as a realistic outcome rather than a failure. The Elliptic dataset already includes handcrafted local and aggregated transaction features, so a feature-based Random Forest can exploit a lot of the graph neighbourhood signal indirectly. GNN message passing adds value compared with plain GCN, but it is not automatically better than a strong feature model.

The per-timestep analysis gives the clearest explanation of the remaining difficulty. TempEdge-ResGNN performs strongly on earlier test timesteps, for example illicit-F1 **0.9140** at timestep 35 and **0.7475** at timestep 42. After timestep 43, performance collapses: F1 is **0.0000** at timestep 43, **0.0267** at timestep 44, and remains very low through the final timesteps. The distribution-shift table shows this issue across all models. TempEdge-ResGNN drops from **0.7410** F1 on timesteps 35-42 to

0.0293 on timesteps 43-49. Random Forest also drops from **0.8992** to **0.0284**. This suggests the issue is not only the GNN design; the test distribution itself changes sharply.

The best TempEdge-ResGNN run catches **719** illicit transactions and misses **364** illicit transactions, with **466** false positives and **15,121** true negatives. This corresponds to precision **0.6068** and recall **0.6639** for the illicit class in that run. For the best overall Random Forest model, the confusion matrix in Figure 9 shows **778** true illicit detections, **305** missed illicit transactions, **70** false positives, and **15,517** true licit predictions. In an AML setting, this trade-off matters because false negatives mean missed illicit activity, while false positives create manual review cost.

Table 1. Main model comparison. Test-set performance of classical baselines, neural baselines, standard GNNs, and TempEdge-ResGNN. Neural models are reported as mean $\pm$ standard deviation over five seeds; classical baselines are single fitted models.

Model	Runs	Illicit precision	Illicit recall	Illicit-F1	Macro-F1	Micro-F1	ROC-AUC	PR-AUC
LogisticRegression	1	0.1405	0.9483	0.2447	0.4953	0.6197	0.8489	0.2044
RandomForest	1	0.9175	0.7184	0.8058	0.8969	0.9775	0.9115	0.7820
MLP	5	0.3983 $\pm$ 0.0246	0.6188 $\pm$ 0.0194	0.4839 $\pm$ 0.0138	0.7185 $\pm$ 0.0089	0.9140 $\pm$ 0.0070	0.8758 $\pm$ 0.0026	0.5074 $\pm$ 0.0202
GCN	5	0.3689 $\pm$ 0.0456	0.4914 $\pm$ 0.0385	0.4182 $\pm$ 0.0208	0.6849 $\pm$ 0.0140	0.9106 $\pm$ 0.0136	0.8403 $\pm$ 0.0033	0.3699 $\pm$ 0.0203
GraphSAGE	5	0.4260 $\pm$ 0.0088	0.6151 $\pm$ 0.0137	0.5033 $\pm$ 0.0076	0.7302 $\pm$ 0.0042	0.9211 $\pm$ 0.0020	0.8801 $\pm$ 0.0032	0.5560 $\pm$ 0.0120
GAT	5	0.4740 $\pm$ 0.0329	0.5313 $\pm$ 0.0252	0.5002 $\pm$ 0.0210	0.7316 $\pm$ 0.0117	0.9309 $\pm$ 0.0053	0.8683 $\pm$ 0.0068	0.5375 $\pm$ 0.0339
GATv2	5	0.5065 $\pm$ 0.0354	0.5295 $\pm$ 0.0422	0.5173 $\pm$ 0.0345	0.7414 $\pm$ 0.0184	0.9358 $\pm$ 0.0048	0.8743 $\pm$ 0.0109	0.5330 $\pm$ 0.0499
TempEdge-ResGNN	5	0.5374 $\pm$ 0.1235	0.6428 $\pm$ 0.0448	0.5760 $\pm$ 0.0672	0.7708 $\pm$ 0.0393	0.9364 $\pm$ 0.0204	0.8922 $\pm$ 0.0069	0.6233 $\pm$ 0.0516

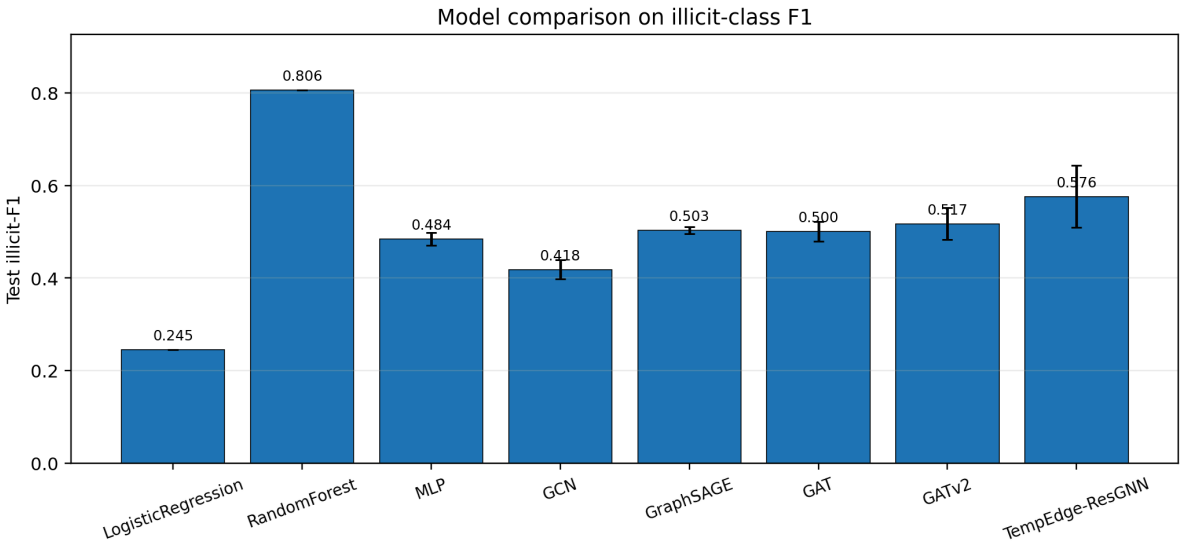


Figure 5. Model comparison by illicit-F1. Random Forest is the best overall model, while TempEdge-ResGNN is the best neural/GNN model.

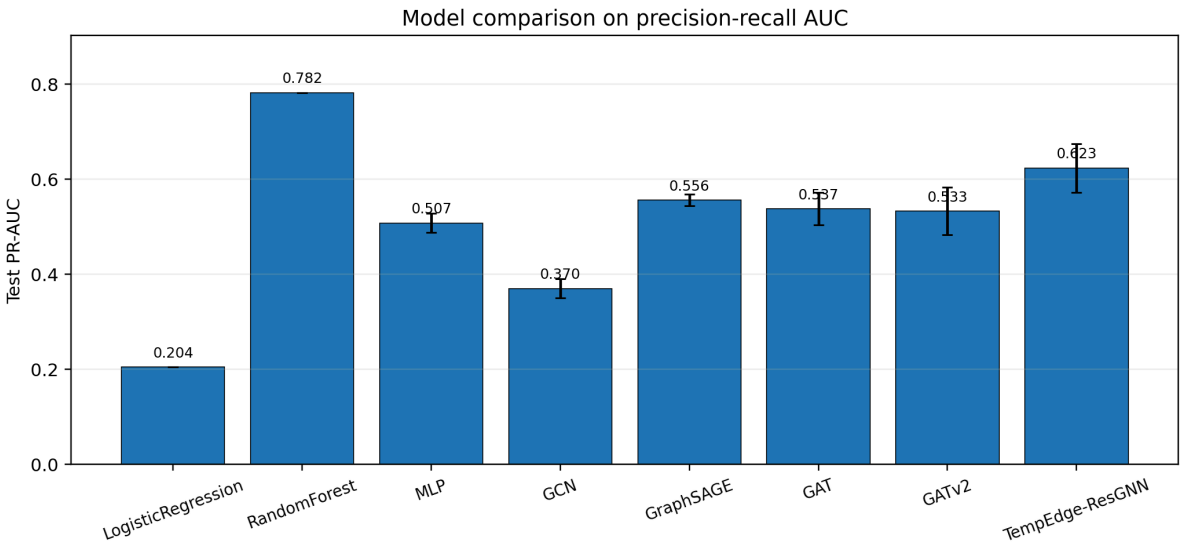


Figure 6. Model comparison by PR-AUC. PR-AUC is important for this imbalanced detection task.

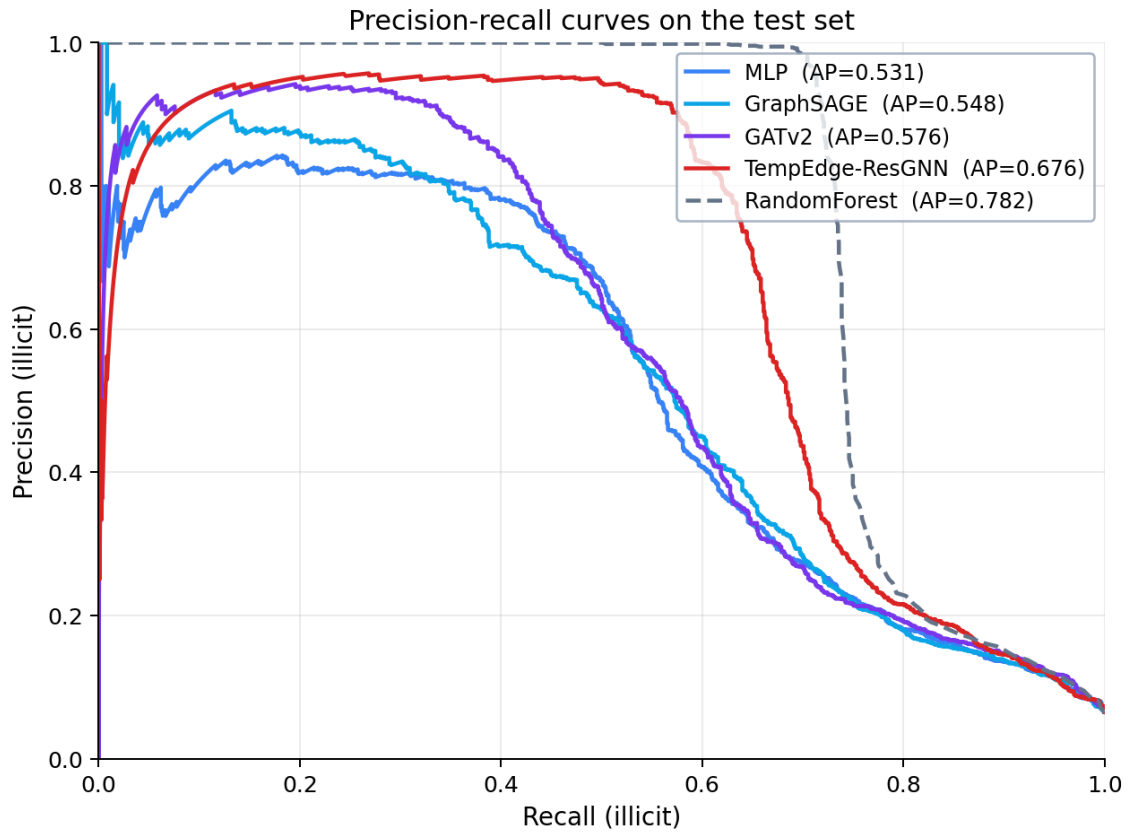


Figure 7. Precision-recall curves. The curves show the threshold trade-off for detecting the minority illicit class.

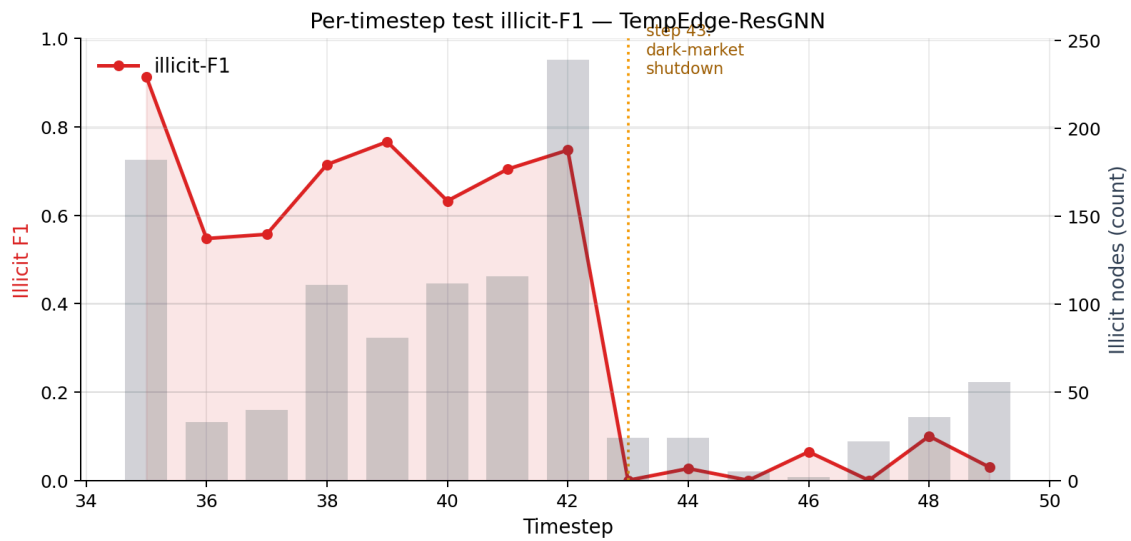


Figure 8. Per-timestep illicit-F1 for TempEdge-ResGNN. Performance is much stronger before timestep 43 than after the temporal distribution shift.

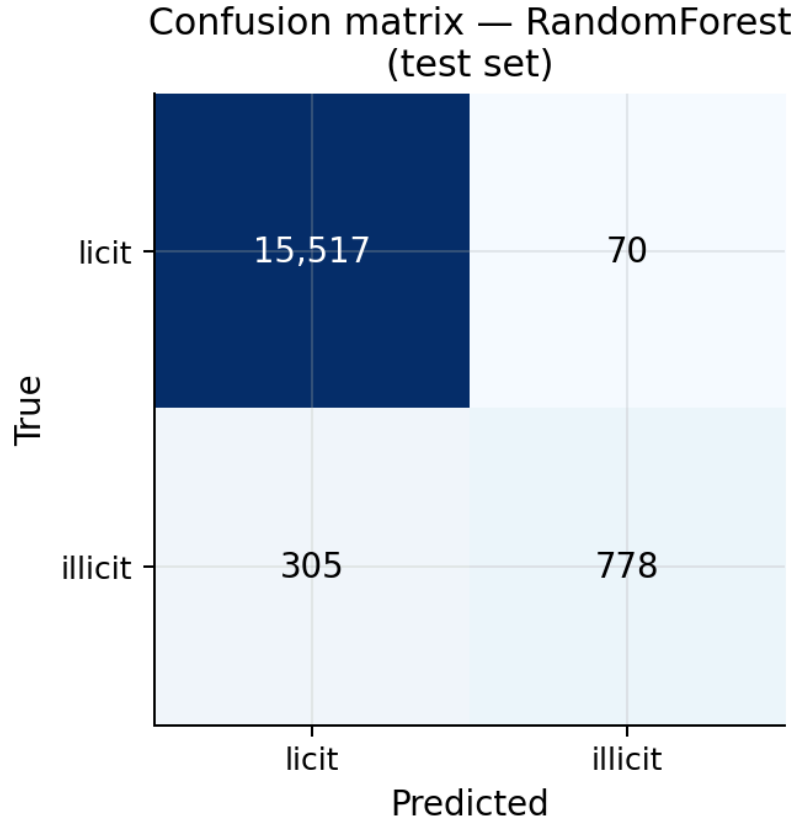


Figure 9. Confusion matrix for Random Forest, the best overall model. This shows why the feature-based baseline remains very strong on the Elliptic handcrafted features.

4. Comprehensive analysis

The ablation study tests which parts of TempEdge-ResGNN actually matter. The full model obtains illicit-F1 **0.5838 ± 0.0440** in the ablation run set. Removing edge attributes lowers F1 to **0.5597 ± 0.0621** , so edge-aware information helps slightly. Removing the residual gate gives **0.5630 ± 0.0423** , also slightly lower than the full model. Removing the non-parametric time features gives **0.5290 ± 0.0323** , which is a larger drop. This supports the idea that time information is important for this dataset.

The strongest ablation result is the old learnable timestep embedding. That version reaches only **0.3351 ± 0.0407** illicit-F1. This makes sense because the training period does not properly train embeddings for later test timesteps. The fixed non-parametric time features are safer because they are defined for all timesteps without needing a separate learned vector for each one.

The ablation also shows that not every added component is clearly useful. Removing JKNet gives **0.5861 ± 0.0464** F1, which is slightly higher than the full model's **0.5838 ± 0.0440** in the ablation table. I therefore do not claim that JKNet improved performance here. It is included as a reasonable design choice, but the evidence says it is optional rather than necessary.

Class-balanced focal loss also does not improve the result. It gives **0.5606 ± 0.0478** F1, lower than the full weighted-BCE model. For this setup, the simpler weighted BCE loss is more stable.

Overall, the comprehensive analysis supports a careful conclusion. TempEdge-ResGNN improves over standard neural/GNN baselines, especially plain GCN, and the ablation shows that non-parametric time features are the most important custom component. However, Random Forest remains the strongest overall model, and temporal distribution shift after timestep 43 is the major limitation.

Table 2. Ablation study. Each row removes or changes one TempEdge-ResGNN design component while keeping the rest of the training setup fixed.

Ablation	Runs	Illicit-F1	PR-AUC	Illicit precision	Illicit recall
A_full	5	0.5838 ± 0.0440	0.6169 ± 0.0340	0.5476 ± 0.0844	0.6345 ± 0.0306
B_no_edge	5	0.5597 ± 0.0621	0.6006 ± 0.0488	0.5223 ± 0.1093	0.6194 ± 0.0385
C_no_gate	5	0.5630 ± 0.0423	0.5746 ± 0.0337	0.5174 ± 0.0816	0.6259 ± 0.0162
D_no_time	5	0.5290 ± 0.0323	0.5286 ± 0.0633	0.5041 ± 0.0379	0.5572 ± 0.0307
E_no_jk	5	0.5861 ± 0.0464	0.6721 ± 0.0357	0.5505 ± 0.0792	0.6332 ± 0.0345
F_cb_focal	5	0.5606 ± 0.0478	0.5888 ± 0.0520	0.5094 ± 0.0859	0.6347 ± 0.0468
G_old_timeembed	5	0.3351 ± 0.0407	0.3416 ± 0.0506	0.2731 ± 0.0676	0.4750 ± 0.1134

Table 3. Temporal distribution-shift breakdown. Illicit-F1 before and after timestep 43. All models suffer a sharp post-43 drop.

Model	F1 steps 35-42	F1 steps 43-49	Drop
LogisticRegression	0.3344	0.0925	0.2419
RandomForest	0.8992	0.0284	0.8708
MLP	0.5916	0.0225	0.5691
GCN	0.4959	0.0509	0.4450
GraphSAGE	0.6049	0.0186	0.5863
GAT	0.6036	0.0069	0.5967
GATv2	0.5755	0.0136	0.5619
TempEdge-ResGNN	0.7410	0.0293	0.7117

Table 4. Best model summary.

Category	Model	Key result
Best overall	RandomForest	Illicit-F1 0.8058; PR-AUC 0.7820
Best neural/GNN	TempEdge-ResGNN	Illicit-F1 0.5760 ± 0.0672; PR-AUC 0.6233 ± 0.0516
Best single TempEdge run	TempEdge-ResGNN seed 42	TP 719, FN 364, FP 466, TN 15,121; precision 0.6068, recall 0.6639

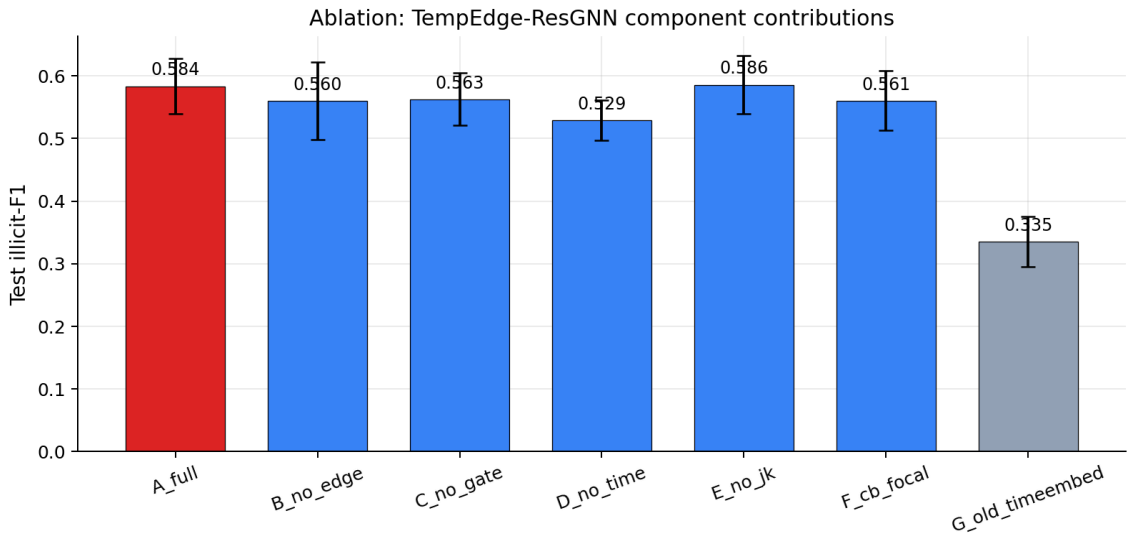


Figure 10. TempEdge-ResGNN ablation by illicit-F1. Non-parametric time features are the most clearly useful component, while JKNet is not clearly beneficial.

Limitations and future work

The first limitation is that this project uses a transductive graph setup. The full graph structure is available during training, although labels are masked. This is acceptable for this assignment and for many PyG-style node-classification experiments, but a stricter deployment setting would require inductive or streaming evaluation where future graph structure is not visible.

The second limitation is that the available edge features are not as rich as real transaction metadata. In this processed graph, all edges are within the same timestep, so edge temporal difference does not capture true transaction delay. A future version could use richer blockchain data with wallet addresses, transaction values, timestamps, or multi-hop address-transaction structure.

The third limitation is that Random Forest remains stronger than the GNNs. A useful next step would be a hybrid model, for example training Random Forest on the original features plus GNN-derived node embeddings. That would test whether the GNN representation adds information beyond the handcrafted features.

Finally, the sharp post-43 collapse suggests that distribution-shift-aware training is needed. Future work could test rolling-window retraining, domain adaptation, temporal calibration, or dynamic graph models such as EvolveGCN or TGN under a carefully controlled evaluation protocol.

Conclusion

This project shows that Bitcoin illicit transaction detection can be formulated naturally as a graph node-classification problem. The Elliptic dataset is a good fit because transactions are connected by payment-flow edges, the labels are highly imbalanced, and the temporal split creates a realistic evaluation setting.

The best overall model is Random Forest, with illicit-F1 **0.8058**. Among neural and graph models, the proposed TempEdge-ResGNN is best, with illicit-F1 **0.5760 ± 0.0672** and PR-AUC **0.6233 ± 0.0516**. The ablation study shows that non-parametric time features are the most important custom component, while JKNet is not clearly beneficial. The per-timestep analysis shows that the main difficulty is temporal distribution shift after timestep 43.

My main conclusion is that GNNs are useful for this AML graph task, but they are not automatically better than strong feature-based baselines. The value of the GNN work is clearest when it is evaluated honestly against multiple baselines and supported by ablation and temporal error analysis.

References

- [1] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, "Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics," KDD Workshop on Anomaly Detection in Finance, 2019.
- [2] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," ICLR, 2017.
- [3] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," NeurIPS, 2017.
- [4] P. Veličković et al., "Graph Attention Networks," ICLR, 2018.
- [5] S. Brody, U. Alon, and E. Yahav, "How Attentive are Graph Attention Networks?" ICLR, 2022.
- [6] K. Xu, C. Li, Y. Tian, T. Sonobe, K. Kawarabayashi, and S. Jegelka, "Representation Learning on Graphs with Jumping Knowledge Networks," ICML, 2018.
- [7] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-Balanced Loss Based on Effective Number of Samples," CVPR, 2019.
- [8] PyTorch Geometric documentation, `torch_geometric.datasets.EllipticBitcoinDataset`. Accessed 31 May 2026.